

Prospective Prediction of OOD Degradation from Source-Side Training Dynamics

Alexander Monin 

*University of South Carolina
712 Main St, 404
Columbia SC, 29208*

E-mail: amonin@mailbox.sc.edu

ABSTRACT: We study whether persistent out-of-distribution (OOD) degradation can be predicted before it is directly observed using only source-side training dynamics. In a controlled shortcut-learning setting, a simple logistic regression predictor develops a clear prospective signal, while training time alone does not. Temporal summaries of the source-side quantities are substantially more informative than their current values. When transferred without additional training from a CNN to an MLP, confidence and entropy dynamics retain substantial predictive information. These results provide a proof of principle that source-side training dynamics can contain an early warning signal for future OOD failure.

KEYWORDS: shortcut learning, training dynamics, OOD degradation, spurious features

Contents

1	Introduction	1
2	Experimental Setup	3
2.1	Shortcut-learning model organism	3
2.2	Persistent degradation event	5
2.3	Prospective prediction task	5
2.4	Trajectory selection	5
2.5	Source-side observables and predictor	6
2.6	Architecture-transfer setting	6
3	Results	7
3.1	Time-only baseline	7
3.2	Prospective predictor training	8
3.3	Metrics and features	11
3.4	Transfer to a different architecture	13
4	Discussion	14
A	Clean-dataset performance	16
B	Additional predictor tests	16

1 Introduction

The rapid progress of AI systems makes questions of safety and reliability increasingly important. For broad discussions of technical AI safety, see [1, 2]. One important source of reliability failures is distribution shift: models that perform well on the training distribution can suffer substantial degradation when evaluated out of distribution (OOD) under realistic shifts [3]. More generally, successful optimization on the training objective does not uniquely determine how a model will behave outside the training distribution. Many different solutions may achieve essentially the same performance on the training and validation data while relying on different predictive rules. This phenomenon was discussed in [4] under the name of underspecification.

A closely related and extensively studied manifestation of this problem is shortcut learning, where a model relies on predictive features that work well on the training distribution but fail under distribution shift [5]. Such behavior has been demonstrated in a variety of settings, including medical imaging, image segmentation, and vision-language representation learning [6–8].

Training dynamics can reveal how such dependencies are acquired. For example, [9] characterizes individual examples using the model’s confidence in the correct class and its variability over training. More directly in the context of shortcut learning, [10] shows that harmful spurious features can be detected by observing the learning dynamics of the early layers of deep networks. The work in [11] further shows that simpler or more strongly correlated spurious features can slow the subsequent learning of core features, and that the learning of spurious and core features does not always separate into distinct stages.

Other approaches address OOD failure in different ways. The OOD accuracy of a model can be estimated without OOD labels by using unlabeled samples from the OOD distribution [12, 13]. Baek et al. also estimate OOD performance across checkpoints along a training trajectory. These methods estimate the model’s OOD performance at the time of evaluation and require access to samples from the OOD distribution. Other methods use the behavior of individual training examples to identify examples that should receive greater emphasis during debiasing [14, 15]. Their goal is to improve robustness by modifying the training procedure.

These approaches address either the detection and mitigation of spurious behavior or the estimation of current OOD performance. This leaves a different prospective question. Detecting that a model has started to use a spurious feature does not determine when its effect will appear as a persistent OOD performance gap. Similarly, estimating the current OOD performance does not predict whether such a degradation will emerge later. We therefore ask whether source-side training dynamics contain information about an approaching OOD degradation before that degradation is directly observed.

In this paper, we study this question in a controlled shortcut-learning setting. We train models on a source distribution containing a perfectly correlated shortcut and evaluate them on a clean OOD distribution in which the shortcut is absent. The clean OOD distribution is used to define the degradation event and in the trajectory collection and selection procedure, but is never provided to the predictor. At each training step, the predictor uses only source-side quantities observed up to that time. For many trajectories, the source and clean OOD accuracies evolve similarly for a substantial initial period, while a large and persistent gap only develops later. We ask whether the source-side information available before this event can be used to predict that the

degradation is approaching.

We find that a simple logistic regression predictor can detect a prospective signal before the degradation event. This signal cannot be explained by training time alone and depends strongly on the recent history of the source-side quantities. For the CNN trajectories, temporal summaries of the training dynamics consistently perform better than the current values alone. Parameter and gradient dynamics alone also retain a prospective signal, showing that the effect is not confined to output-level performance statistics.

Finally, we test whether the same signal survives a change of model architecture. We train the predictor on the CNN trajectories and apply it without any additional training to an MLP with different degradation dynamics. The predictor using the complete set of source-side quantities transfers only weakly. However, when we restrict the predictor to confidence and entropy dynamics, a substantial predictive signal remains. This provides a proof of principle that *source-side training dynamics can contain information about an upcoming OOD failure before this failure is directly observed*.

2 Experimental Setup

2.1 Shortcut-learning model organism

We use a binary version of MNIST as a controlled shortcut-learning environment, assigning label 0 to digits 0-4 and label 1 to digits 5-9. The images are converted to RGB and a single label-correlated colored pixel is added in the upper-left corner. The resulting source distribution has perfect shortcut-label correlation. The OOD distribution consists of the corresponding clean images with the shortcut removed.

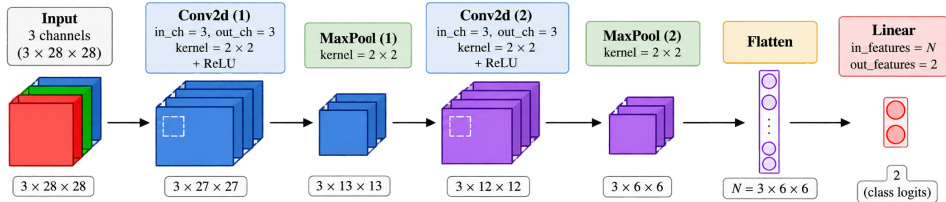


Figure 1. Architecture of the CNN used as the main model organism.

Our main model architecture is a small convolutional neural network (CNN), see Figure 1. It consists of two convolutional blocks, Conv2d + ReLU + MaxPool2d, followed

by two fully connected layers. Both convolutional layers have three output channels, and all convolutional and max-pooling kernels have size 2×2 . The hidden fully connected layer has dimension 4, and the output layer has dimension 2.

The main training parameters are summarized in Table 1. Before introducing the shortcut, we verify that the architecture reaches at least 95% accuracy on the clean dataset within $T_{\max} = 12\,000$ training steps, corresponding to 10 epochs, see Appendix A. We then train multiple models with different random seeds on the shortcut dataset for at most $T_{\max}$ steps. A run is stopped earlier if, for 100 consecutive evaluation steps, $A_{\text{source}} > 0.99$ and $A_{\text{source}} - A_{\text{clean}} > 0.45$.

Training parameter	Value
Batch size B	50
Optimizer	Adam
Learning rate	10^{-3}
(β_1, β_2)	(0.9, 0.999)
Loss	Cross entropy
Maximum training steps $T_{\max}$	12 000

Table 1. Training parameters used throughout the experiments.

For each training trajectory, corresponding to a different random seed, we record the nine source-side quantities listed in Table 2.

Observable	Definition
Training loss	Cross-entropy loss on the current batch
Training accuracy	Accuracy on the current batch
Gradient norm	$\ \vec{g}_t\ $
Parameter norm	$\ \vec{\theta}_t\ $
Local parameter change	$\ \vec{\theta}_t - \vec{\theta}_{t-1}\ $
Global parameter change	$\ \vec{\theta}_t - \vec{\theta}_0\ $
Confidence	Mean maximum class probability
Confidence correct	Mean maximum class probability on correct predictions
Entropy	Mean prediction entropy

Table 2. Source-side quantities recorded during training. Here $\vec{\theta}_t$ and $\vec{g}_t$ denote the concatenated parameter and gradient vectors at training step t .

2.2 Persistent degradation event

We evaluate the source and clean accuracies after every training step and define the degradation by

$$D(t) = |A_{\text{source}}(t) - A_{\text{clean}}(t)|. \quad (2.1)$$

The degradation time T_{deg} is the first time for which $D(T_{\text{deg}}) > \delta_{\text{deg}} = 0.05$ and at least 81 of the following 90 steps also satisfy this condition. This persistence requirement suppresses transient fluctuations. Information from the clean dataset after the candidate degradation time is used only offline to determine T_{deg} and is never provided to the predictor.

2.3 Prospective prediction task

Our main goal is to predict the degradation event using source-side training dynamics only. At every training step t , the predictor uses the preceding $W = 100$ training steps, including the current step, and predicts whether persistent degradation will occur within a horizon $H = 200$. A prediction at time t is therefore considered positive when

$$0 < T_{\text{deg}} - t \leq H. \quad (2.2)$$

2.4 Trajectory selection

Beyond the degradation-event definition and the stopping criterion described above, the OOD information is used here only to select trajectories for predictor training. Different random seeds can lead to substantially different OOD behavior, as illustrated in Figure 2, see also [4]. We retain trajectories that reach $A_{\text{source}} > 0.75$, satisfy the persistent degradation criterion defining T_{deg} , and have at least $S_{\text{pre}} = W + 2H = 500$ pre-degradation steps. This allows us to construct a balanced set of positive windows within H steps of T_{deg} and equally many earlier negative windows.

Trajectories that degrade too early are excluded as fast-degradation runs. Runs whose source accuracy does not exceed $A_{\text{stalled}} = 0.54$ are excluded as stalled. Trajectories with no observed degradation are not used to train the predictor, but are retained for the full-history evaluation of false warnings. For these trajectories, we include only windows satisfying $t + H \leq T_{\text{end}}$.

The degrading trajectories are split into training and test sets before individual windows are extracted. Thus, no trajectory contributes windows to both sets. Consecutive windows share $W - 1$ of their W measurements and are therefore strongly correlated, so the trajectory is the relevant independent unit when comparing predictor variants.

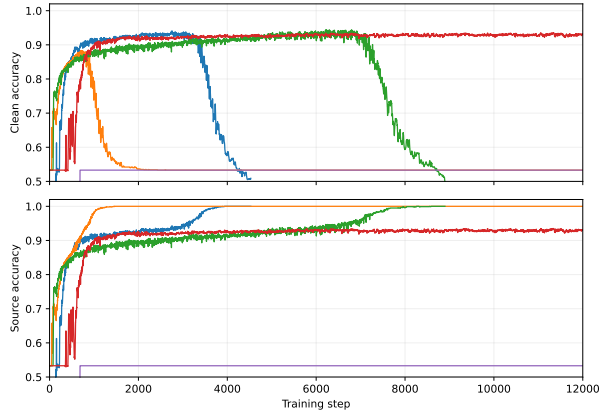


Figure 2. Sample CNN training trajectories for different random seeds. Shown are the source and clean accuracies.

2.5 Source-side observables and predictor

For each source-side quantity, we summarize the preceding $W = 100$ steps by its current value, mean, standard deviation, and slope. With nine source-side quantities, each window is represented by 36 features. Each feature is normalized using the mean and standard deviation computed on the predictor training set, and the same normalization is used for all test datasets.

We use a logistic regression predictor. If $\vec{x}(t)$ denotes the normalized feature vector for the window ending at time t , the predicted probability is

$$p(t) = \sigma(\vec{w} \cdot \vec{x}(t) + b). \quad (2.3)$$

The predictor is trained on the positive and negative windows defined above using binary cross-entropy with logits and Adam with learning rate 10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.999$, for 7500 optimization steps. For the classification metrics below and for the fraction of flagged trajectories shown in the figures, we classify a window as positive when $p(t) > 0.5$.

2.6 Architecture-transfer setting

To test whether the prospective signal survives a change of architecture, we also consider a small MLP with two fully connected hidden layers of dimensions 4 and 2. As for the CNN, we verify that the architecture can reach approximately 95% accuracy on the clean dataset, see Appendix A.

The MLP trajectories show qualitatively different degradation dynamics from the CNN trajectories, as illustrated in Figure 3. We use the same operational definition

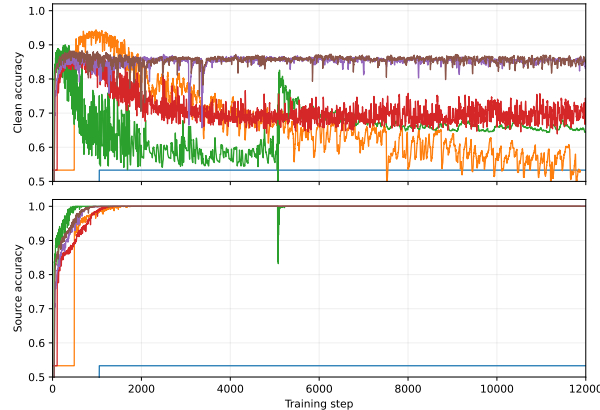


Figure 3. Sample training trajectories for the MLP trained on the shortcut dataset. Different random seeds lead to different training behavior. Shown are the clean accuracy and the source accuracy.

of the degradation event. For the MLP, this criterion generally identifies a persistent separation between the source and clean accuracies rather than a late collapse of the clean accuracy. The predictor trained on the CNN trajectories is then applied directly to the MLP trajectories without retraining.

3 Results

In this section we present the main results. We first study prospective prediction within the CNN architecture and then test whether the learned signal generalizes to the MLP architecture without any additional retraining.

3.1 Time-only baseline

Before considering the source-side training dynamics, we ask whether the degradation event can be predicted from the training time alone. We train the same logistic predictor on the same balanced prediction dataset, using only the training step t as input. This controls for the possibility that degradation simply tends to occur at a characteristic stage of training.

The result is shown in Figure 4. The time-only predictor does not distinguish windows far from and close to the degradation event. This is already the case on the trajectories used to train the predictor, and the behavior remains essentially unchanged on the held-out test trajectories. Thus, training time alone does not provide a useful prospective signal.

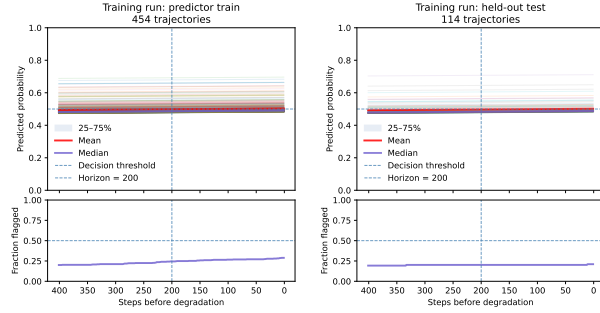


Figure 4. Performance of the time-only predictor on the balanced prediction dataset. Shown are the predictor-training trajectories and the held-out test trajectories. The upper panels show the predicted probability as a function of the number of training steps before degradation, and the lower panels show the corresponding fraction of flagged trajectories.

3.2 Prospective predictor training

We now turn to the source-side training dynamics. We train the logistic regression predictor on the balanced prediction dataset constructed as described above. Figure 5 shows its performance on the predictor-training and held-out test trajectories from the same CNN run, together with two independently generated CNN runs.

We also examine the predictor over a wider part of the pre-degradation trajectory. At larger lead times fewer trajectories contribute to the aggregate statistics, making the curves progressively noisier. We therefore restrict this comparison to the last 800 training steps before degradation. At this lead time approximately 40%-45% of the degrading trajectories still contribute in each of the three CNN runs. Figure 6 shows the predictor over this wider range. For the training run, this extended diagnostic includes all qualifying degrading trajectories, including those used to train the predictor. The two independent CNN runs provide out-of-sample comparisons.

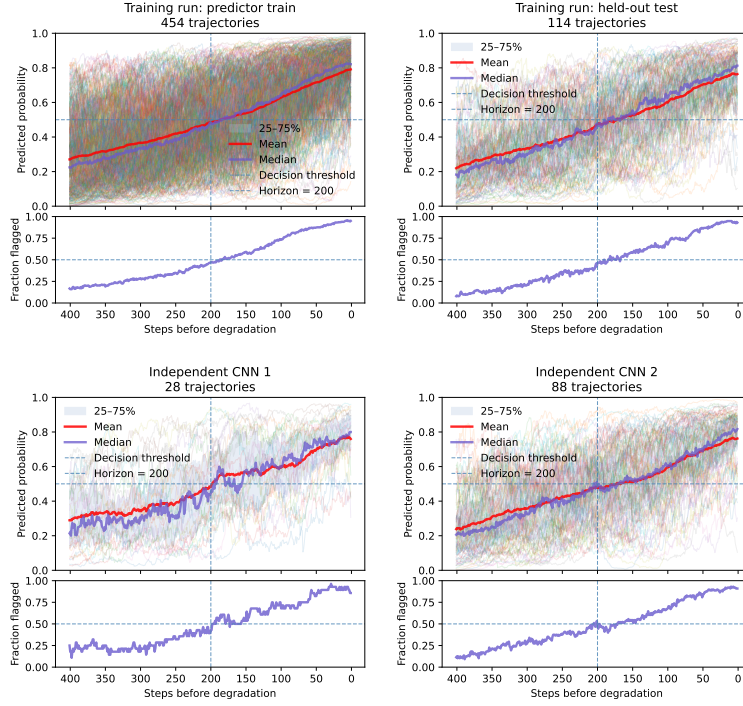


Figure 5. Performance of the logistic regression predictor on the balanced prediction datasets. Shown are the predictor-training and held-out test trajectories from the training run, together with two independently generated CNN runs. The upper panels show the predicted probability as a function of the number of training steps before degradation, and the lower panels show the corresponding fraction of flagged trajectories.

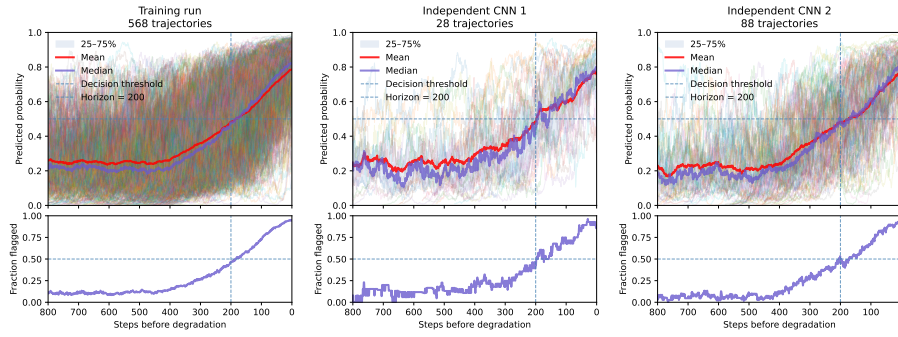


Figure 6. Performance of the same trained predictor over the last 800 training steps before degradation. Shown are all qualifying degrading trajectories from the training run, including those used to train the predictor, together with the same two independently generated CNN runs as in Figure 5. The upper panels show the predicted probability as a function of the number of training steps before degradation, while the lower panels show the corresponding fraction of flagged trajectories.

3.3 Metrics and features

We next investigate which source-side metrics and which features of their recent history are important for prospective prediction. Before comparing different choices, there is one immediate concern to address. Neither the source evaluation accuracy nor the clean evaluation accuracy is provided to the predictor, with the latter used only to define the degradation event. The predictor does, however, have access to the training loss and training accuracy. Since averaging the training accuracy over the prediction window strongly suppresses its batch-to-batch fluctuations, the predictor could in principle exploit a smoothed performance signal. For $B = 50$ and $W = 100$, the corresponding binomial standard deviation is at most approximately 0.007.

We therefore perform a direct test by removing both the training loss and training accuracy from the predictor inputs. The results are summarized in Table 3. The predictive performance decreases only moderately and remains well above the time-only baseline. Thus, the prospective signal is not solely driven by the training loss and training accuracy as explicit inputs. This does not mean that all performance-related information has been removed. In binary classification, the remaining output statistics are themselves derived from the predicted class probabilities and can therefore retain information correlated with loss and accuracy, although this information is compressed by averaging over the batch.

Predictor inputs	Training-run test	Independent CNN 1	Independent CNN 2
All, four features	0.74	0.72	0.71
All, mean	0.71	0.70	0.70
All, current	0.63	0.61	0.63
No loss/accuracy	0.71	0.69	0.67
Output statistics	0.66	0.67	0.64
Parameter/gradient	0.65	0.63	0.67

Table 3. Prediction accuracy on the balanced prediction datasets for different choices of source-side metrics and temporal features. The datasets contain 114, 28, and 88 trajectories for the Training-run test set, Independent CNN 1, and Independent CNN 2, respectively. “All” denotes all source-side metrics. “Four features” denotes the current value, mean, standard deviation, and slope over the preceding window. Unless stated otherwise, the four temporal features are used.

We next consider the role of the temporal features. Keeping all source-side metrics, the current value alone performs substantially worse than the mean over the preceding window. Adding the current value, standard deviation, and slope gives a further but

smaller improvement. Thus, most of the improvement in balanced prediction accuracy comes from incorporating the recent history through the window mean.

The same distinction is particularly pronounced on trajectories with no observed degradation. Since all eligible windows on these trajectories are negative, they provide a direct test of false warnings. Table 4 shows the mean trajectory-level false-positive rate for the same three choices of temporal features. For each trajectory, the false-positive rate is the fraction of eligible windows classified as positive, and we then average this quantity across trajectories.

Features	Training run	Independent CNN 1	Independent CNN 2
Four features	0.03	0.03	0.03
Mean	0.09	0.12	0.13
Current	0.75	0.82	0.77

Table 4. Mean trajectory-level false-positive rates on trajectories with no observed degradation for different temporal features. The training run, Independent CNN 1, and Independent CNN 2 contain 84, 6, and 10 such trajectories, respectively. For each trajectory, the false-positive rate is the fraction of eligible windows classified as positive, and the reported value is the mean across trajectories.

Using only the current values leads to false-positive rates between 0.75 and 0.82. Using the window mean reduces them to approximately 0.09-0.13, while using all four temporal features reduces them further to approximately 0.03. Thus, across all three CNN cohorts, temporal averaging greatly reduces false positives relative to the current-value predictor, and the additional temporal features reduce them further relative to the mean alone.

Finally, we separate the remaining metrics into two restricted groups. The output statistics, consisting of confidence, confidence on correctly classified examples, and entropy, and the parameter and gradient quantities both retain substantial predictive information, as shown in Table 3. Since the parameter and gradient group contains no output-level quantities, this shows that, within the CNN experiments, the prospective signal is not confined to output-level performance statistics. Their relative ordering changes between datasets, so we do not interpret one group as systematically more informative than the other. Neither group reproduces the performance obtained when all source-side metrics are combined. The corresponding prediction curves for the different choices of temporal features and source-side metrics on the balanced prediction datasets are shown in Appendix B.

3.4 Transfer to a different architecture

We next test whether the prospective signal identified in the CNN trajectories survives a change of architecture. As discussed above, the MLP exhibits qualitatively different degradation dynamics. We use the same operational definition of the degradation event, which for the MLP generally identifies a persistent separation between the source and clean accuracies rather than a late collapse of the clean accuracy. In all cases, the predictor and its normalization are trained only on the CNN trajectories and are then applied directly to the MLP trajectories without additional training.

We first use the complete set of source-side quantities and the same four temporal features as in the main CNN analysis. The prediction accuracy decreases from 0.74 on the held-out CNN trajectories to 0.58 on the balanced MLP dataset. As shown in the left panel of Figure 7, the predicted probability still tends to increase as the degradation event is approached, but the separation is substantially weaker than for the CNN trajectories. Thus, the complete predictor transfers only weakly to the MLP.

We then consider the two restricted metric groups introduced in Section 3.3. The parameter and gradient quantities depend directly on the architecture and parameterization of the model, so their numerical scales and training dynamics can change substantially between the CNN and MLP. Consistent with this, the predictor restricted to these quantities performs poorly on the MLP, with the prediction accuracy decreasing from 0.65 on the held-out CNN trajectories to 0.38 on the MLP.

The output statistics, consisting of confidence, confidence on correctly classified examples, and entropy, have the same interpretation for both architectures and do not depend directly on the parameterization of the network. The predictor restricted to these quantities transfers substantially better. Its prediction accuracy is 0.66 on the held-out CNN trajectories and 0.65 on the MLP. As shown in the right panel of Figure 7, the predicted probability also increases systematically as the degradation event is approached. The corresponding prediction accuracies, together with the mean-only and time-only comparisons, are summarized in Table 5.

These results show that the source-side quantities are not equally transferable across architectures. The architecture-dependent parameter and gradient quantities transfer poorly, while the output statistics retain substantial predictive information across the change of architecture. Table 5 also shows that, for the predictor using all source-side quantities, using only the window mean gives a somewhat higher MLP prediction accuracy, 0.61 compared with 0.58 for the four-feature predictor.

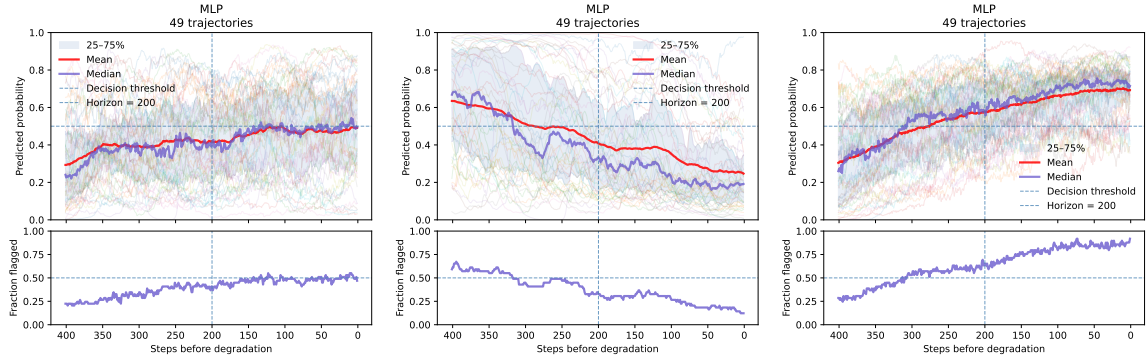


Figure 7. Transfer of CNN-trained predictors to MLP trajectories without additional training. Left: predictor using all source-side quantities and the four temporal features. Center: predictor using only the parameter and gradient quantities. Right: predictor using only the output statistics. The complete predictor transfers only weakly, the parameter and gradient predictor transfers poorly, and the output statistics retain a substantially clearer prospective signal.

Predictor	CNN held-out	MLP
All metrics, four features	0.74	0.58
All metrics, mean	0.71	0.61
Output statistics, four features	0.66	0.65
Parameter/gradient, four features	0.65	0.38
Time only	0.50	0.50

Table 5. Transfer of CNN-trained predictors to the MLP architecture. The predictors and their normalization are trained only on CNN trajectories and are applied to the MLP trajectories without additional training. The balanced evaluation datasets contain 114 held-out CNN trajectories and 49 MLP trajectories. Reported values are prediction accuracies on these datasets.

4 Discussion

The main result of this work is that persistent OOD degradation can be preceded by a detectable change in the source-side training dynamics. In our setting, a predictor using only information available up to the current training step can distinguish windows that are approaching the degradation event from windows that are still far from it. A time-only predictor does not reproduce this behavior, indicating that the signal is contained in the state and recent dynamics of the model rather than simply in the progress of training.

The temporal structure of the source-side quantities is important. Predictors using only their current values perform substantially worse than predictors that summarize their recent history. The window mean already captures most of the improvement in balanced prediction accuracy, while adding the current value, standard deviation, and slope gives only a modest further improvement. The clearer advantage of using all four temporal features appears on trajectories with no observed degradation, where they consistently produce fewer false positives than the mean alone.

Within the CNN experiments, the prospective signal is not confined to output-level statistics. The parameter and gradient quantities alone retain substantial predictive power. Removing the training loss and training accuracy also leaves a substantial signal, although the remaining output statistics can still contain loss-related information. The transfer experiment shows that the source-side quantities are not equally transferable across architectures. The full CNN predictor transfers poorly to the MLP, while the output statistics retain substantial predictive information across architectures.

There are several important limitations. The present experiments use small neural networks and a deliberately simple shortcut in binary MNIST that is perfectly correlated with the label. Whether similar prospective signals persist when this correlation is imperfect remains to be determined. In addition, the degradation time is defined operationally through the threshold $\delta_{\text{deg}} = 0.05$, so the timing of the prediction signal is relative to this definition. Choosing a smaller threshold would generally move T_{deg} earlier. The degradation event is also defined using performance on a clean OOD distribution. This information is used to define the event and select trajectories, but is never provided to the predictor. We therefore do not claim that the same source-side quantities will provide useful warning signals for arbitrary distribution shifts or larger models. Rather, the experiments establish in a controlled setting that *source-side training dynamics can* contain information about a future degradation event before that event becomes visible in OOD performance.

A further limitation is that the MLP cohort contains no trajectories without observed degradation. The transfer experiment therefore tests whether the prospective signal survives a change of architecture. It does not allow us to establish a calibrated decision threshold or to estimate the false-positive rate on MLP trajectories with no observed degradation.

An important next step is to determine whether similar prospective signals can be identified in larger models and in settings where the shortcut and the relevant distribution shift are not specified in advance. Turning these prospective probabilities into an operational alarm is a separate problem that requires additional choices about acceptable warning times and false warnings, which we leave to future work.

A Clean-dataset performance

As a basic capacity check, before studying shortcut learning we verify that both model architectures are capable of learning the underlying classification task in the absence of the shortcut. Figure 8 shows representative clean-training trajectories for the CNN and MLP. These experiments are only used to establish that both architectures have sufficient capacity to solve the clean task.

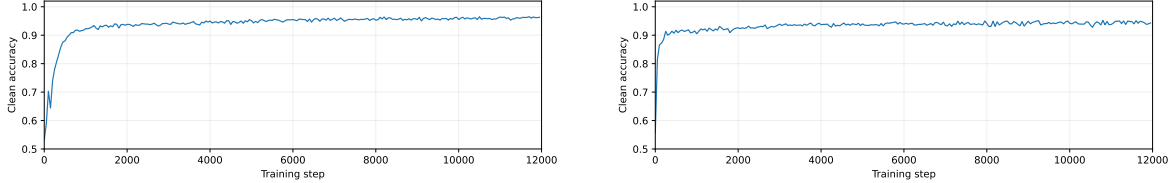


Figure 8. Representative training trajectories on the clean dataset for the CNN (left) and MLP (right). The CNN reaches at least 95% accuracy within the maximal training time $T_{\max} = 12\,000$ steps, while the MLP reaches approximately 95%.

B Additional predictor tests

In this section we present the prediction curves corresponding to the different choices of temporal features and source-side metrics discussed in Section 3.3. The numerical comparison is given in the main text. Here we show the corresponding predictor behavior on the balanced prediction datasets.

We first test the effect of removing the explicit training loss and training accuracy from the predictor inputs. Figure 9 shows the result after removing both the training loss and training accuracy from the predictor inputs while retaining the four temporal features for all remaining source-side metrics.

We next compare different ways of using the recent training history while keeping all source-side metrics. Figure 10 compares the predictor using only the current value of each metric with the predictor using only the mean over the preceding window. As discussed in the main text, averaging over the recent history substantially improves the prediction performance.

Finally, we separate the remaining observables into two broad groups. Figure 11 compares the predictor restricted to the parameter- and gradient-based quantities with the predictor restricted to the output statistics, consisting of confidence, confidence on correctly classified examples, and entropy.

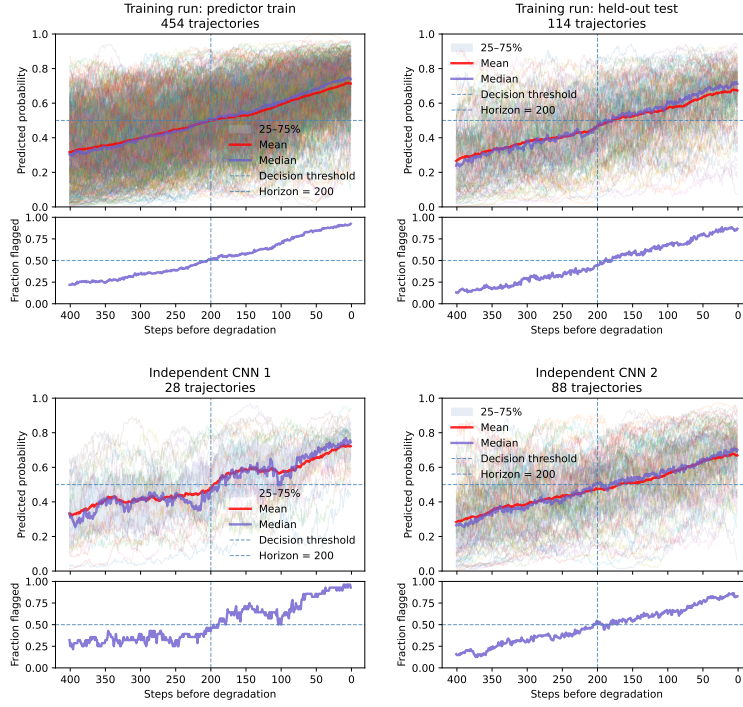


Figure 9. Performance on the balanced prediction datasets after removing the training loss and training accuracy from the predictor inputs while retaining the four temporal features for the remaining source-side metrics.

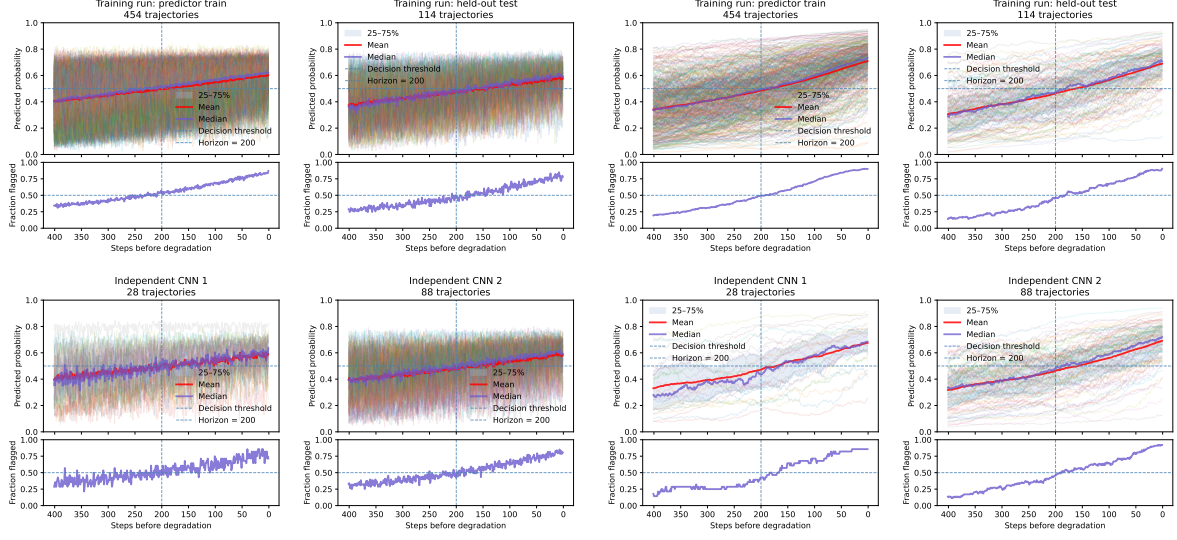


Figure 10. Comparison of different temporal features on the balanced prediction datasets. Left: predictor using only the current value of each source-side metric. Right: predictor using only the mean over the preceding window.

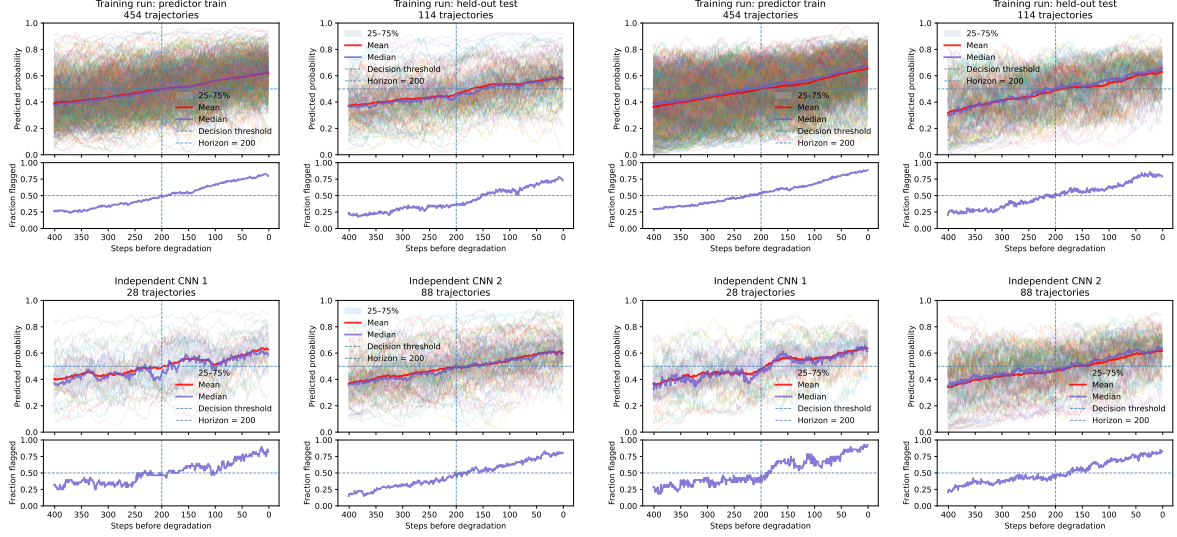


Figure 11. Comparison of different groups of source-side metrics on the balanced prediction datasets, using all four temporal features. Left: parameter- and gradient-based quantities. Right: output statistics consisting of confidence, confidence on correctly classified examples, and entropy.

References

- [1] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman and D. Mané, *Concrete Problems in AI Safety*, *arXiv preprint arXiv:1606.06565* (2016) [[1606.06565](#)].
- [2] D. Hendrycks, N. Carlini, J. Schulman and J. Steinhardt, *Unsolved Problems in ML Safety*, *arXiv preprint arXiv:2109.13916* (2021) [[2109.13916](#)].
- [3] P.W. Koh, S. Sagawa, H. Marklund, S.M. Xie, M. Zhang, A. Balsubramani et al., *WILDS: A Benchmark of in-the-Wild Distribution Shifts*, in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139 of *Proceedings of Machine Learning Research*, pp. 5637–5664, PMLR, 2021, <https://arxiv.org/abs/2012.07421> [[2012.07421](#)].
- [4] A. D’Amour, K. Heller, D. Moldovan, B. Adlam, B. Alipanahi, A. Beutel et al., *Underspecification Presents Challenges for Credibility in Modern Machine Learning*, *Journal of Machine Learning Research* **23** (2022) 1 [[2011.03395](#)].
- [5] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge et al., *Shortcut Learning in Deep Neural Networks*, *Nature Machine Intelligence* **2** (2020) 665 [[2004.07780](#)].
- [6] A. Brown, N. Tomasev, J. Freyberg, Y. Liu, A. Karthikesalingam and J. Schrouff, *Detecting shortcut learning for fair medical AI using shortcut testing*, *Nature Communications* **14** (2023) 4314 [[2207.10384](#)].
- [7] M. Lin, N. Weng, K. Mikolaj, Z. Bashir, M.B.S. Svendsen, M.G. Tolsgaard et al., *Shortcut Learning in Medical Image Segmentation*, in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*, vol. 15008 of *Lecture Notes in Computer Science*, pp. 623–633, Springer, 2024, DOI [[2403.06748](#)].
- [8] M. Bleeker, M. Hendriksen, A. Yates and M. de Rijke, *Demonstrating and Reducing Shortcuts in Vision-Language Representation Learning*, *Transactions on Machine Learning Research* **2024** (2024) [[2402.17510](#)].
- [9] S. Swayamdipta, R. Schwartz, N. Lourie, Y. Wang, H. Hajishirzi, N.A. Smith et al., *Dataset Cartography: Mapping and Diagnosing Datasets with Training Dynamics*, in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 9275–9293, Association for Computational Linguistics, 2020, DOI [[2009.10795](#)].
- [10] N. Murali, A.M. Puli, K. Yu, R. Ranganath and K. Batmanghelich, *Beyond Distribution Shift: Spurious Features Through the Lens of Training Dynamics*, *Transactions on Machine Learning Research* (2023) [[2302.09344](#)].

- [11] G. Qiu, D. Kuang and S. Goel, *Complexity Matters: Feature Learning in the Presence of Spurious Correlations*, in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 41658–41697, PMLR, 2024, <https://arxiv.org/abs/2403.03375> [2403.03375].
- [12] S. Garg, S. Balakrishnan, Z.C. Lipton, B. Neyshabur and H. Sedghi, *Leveraging Unlabeled Data to Predict Out-of-Distribution Performance*, in *International Conference on Learning Representations*, 2022, <https://arxiv.org/abs/2201.04234> [2201.04234].
- [13] C. Baek, Y. Jiang, A. Raghunathan and J.Z. Kolter, *Agreement-on-the-Line: Predicting the Performance of Neural Networks under Distribution Shift*, in *Advances in Neural Information Processing Systems*, vol. 35, pp. 19274–19289, 2022, DOI [2206.13089].
- [14] J. Nam, H. Cha, S. Ahn, J. Lee and J. Shin, *Learning from Failure: De-biasing Classifier from Biased Classifier*, in *Advances in Neural Information Processing Systems*, vol. 33, pp. 20673–20684, 2020, <https://arxiv.org/abs/2007.02561> [2007.02561].
- [15] E.Z. Liu, B. Haghighi, A.S. Chen, A. Raghunathan, P.W. Koh, S. Sagawa et al., *Just Train Twice: Improving Group Robustness without Training Group Information*, in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139 of *Proceedings of Machine Learning Research*, pp. 6781–6792, PMLR, 2021, <https://arxiv.org/abs/2107.09044> [2107.09044].